



université
PARIS-SACLAY

FACULTÉ
DES SCIENCES
D'ORSAY

AIRBUS

RAPPORT DE STAGE

Accélération de la résolution des équations de Goldstein par des méthodes d'apprentissage profond

rédigé par :

Joel Pascal SOFFO WAMBO

Etudiant en

MASTER 2 DATA SCIENCE

Stage effectué du 11/04/2022 au 30/09/2022

Directeur de stage : Antoine BENSALAH, Airbus S.A.S

Conseiller de stage : Christine KERIBIN, Université Paris-Saclay

AIRBUS

Sujet de stage

Accélération de la résolution des équations de Goldstein par des méthodes d'apprentissage profond

Rédigé par :

Joël Pascal SOFFO WAMBO

Nombre de pages : 42

Résumé

L'apprentissage profond connaît un grand essor, notamment dans les domaines de la vision par ordinateur et du traitement de langages. Plus récemment, ce type de méthodes a été utilisé pour résoudre des équations aux dérivées partielles (EDPs), notamment avec l'introduction des réseaux de neurones informés de la physique (PINNs), puis d'une nouvelle famille de réseaux de neurones appelés opérateurs neuronaux, permettant d'apprendre directement un opérateur entre espaces de fonctions. Parmi les familles d'EDPs, nous avons les équations de Goldstein, très utilisées en dynamique des fluides, notamment en aérodynamique. Ces équations étant très difficile à résoudre par des méthodes classiques d'analyse numérique, on peut envisager de les résoudre par apprentissage. Pour atteindre cet objectif, on se focalise dans ce stage sur la résolution de cas simplifiés de l'équation de Goldstein (1D et 2D), à l'aide de différentes méthodes d'apprentissage profond.

Abstract

Deep Learning has given tremendous results in the fields of computer vision and natural language processing. More recently, it has also been used to solve partial differential equations (PDEs), notably with the introduction of physics-informed neural networks (PINNs), then of a new family of neural networks called neural operators, making it possible to directly learn an operator between infinite dimensional function spaces. Among the families of PDEs, we have the Goldstein equations, widely used in fluid dynamics, especially in aerodynamics. Since these equations are very difficult to solve by conventional methods of numerical analysis, one could consider solving them using Deep Learning methods. To achieve this, we focus in this internship on solving simplified cases of the Goldstein equation (1D and 2D), using different deep learning methods.

Table des matières

1	L'entreprise	6
1.1	Présentation de l'entreprise	6
1.2	Présentation du laboratoire d'accueil	7
2	Introduction générale	8
2.1	Contexte	8
2.2	Problématique	8
2.3	Objectifs	9
3	Machine Learning pour les EDPs	11
4	Le problème d'apprentissage	13
4.1	Equation de transport : Cas 1D	13
4.1.1	Cas m_0 constant	13
4.1.2	Cas m_0 non constant	13
4.2	Equation de transport : Cas 2D	14
4.3	Bases de données	14
4.4	Fonction de perte	17
5	Travail réalisé : Les opérateurs neuronaux (NO)	19
5.1	Opérateur neuronal de Fourier (FNO)	21
5.1.1	Présentation	21

5.1.2	Résultats	23
5.1.3	Limites du FNO	27
5.2	Réseau à noyau graphique (GNO)	27
5.2.1	Présentation	27
5.2.2	Résultats	28
5.2.3	Commentaires	28
5.3	Opérateur neuronal informé de la physique (PINO)	29
5.3.1	Les PINNs	29
5.3.2	Couplage	30
5.3.3	Résultats	32
6	Bilan général	34
7	Conclusion	35
8	Annexe	39

Table des figures

4.1	Observations tirées de db1	16
4.2	Observations tirées de db2	16
4.3	Observations tirées de db3	18
5.1	Architecture du FNO	22
5.2	Résultats FNO 1D - Cas de db1	24
5.3	Minimisation de la fonction de perte L2 relative - 1D	25
5.4	Résultats FNO 2D	26
5.5	Architecture des PINNs	31
8.1	Ecoulement	40
8.2	Rectified Linear Unit (ReLU)	41
8.3	Gaussian Error Linear Unit (GELU)	41

Remerciements

Je tiens à remercier tout d'abord mon encadrant, Antoine Bensalah, pour le suivi permanent tout au long du stage. Nos discussions et partages m'ont permis de beaucoup mieux comprendre le sujet, et de mieux l'appréhender.

J'aimerais ensuite remercier toute l'équipe d'Issy, pour les discussions et réflexions pendant les repas, et les fous rires au quotidien.

Un merci **très particulier** à mes parents, pour leurs conseils et leur soutien indéfectible depuis le *jour 1*.

J'aimerais enfin terminer en remerciant la Fondation Mathématique Jacques Hadamard (FMJH), pour m'avoir permis d'effectuer ces deux dernières années dans de très bonnes conditions.

Chapitre 1

L'entreprise

1.1 Présentation de l'entreprise

Airbus est une coopération industrielle européenne, spécialisée dans le secteur de l'aéronautique, et dont le quartier général est situé à Blagnac, dans la banlieue de Toulouse. Principalement connue comme étant la référence en matière de construction d'avions de ligne et d'hélicoptères, l'entreprise est aussi fortement présente dans des secteurs tels que :

- **La sécurité**

Airbus propose diverses solutions de lutte contre des menaces telles que les actes de terrorisme, l'espionnage, les cyber-attaques, etc ... à travers des systèmes de cyber-sécurité, des avions tactiques, des hôpitaux mobiles et autres.

- **La défense**

L'entreprise participe à renforcer les défenses gouvernementales, aussi bien par la conception de systèmes de combat tels que des avions de chasse, drones, etc ... que par des systèmes intelligents.

- **L'espace**

Airbus participe à la découverte de destinations inexplorées de notre univers à travers le développement de composantes électroniques et technologies intégrées à des fusées, la construction de satellites, etc ...

A travers ses trois divisions, *Commercial Aircraft (Airbus S.A.S)*, *Defence and Space* et *Helicopters*, Airbus opère à travers le monde entier, et a su au fil du temps, se forger une place en tant qu'acteur indispensable à la construction d'un futur meilleur.

1.2 Présentation du laboratoire d'accueil

Le **CRT**(***Central Research and Technology***), est la division de Recherche et Technologie d'Airbus, chargée de développer de nouvelles technologies innovantes, qui pourront être utilisées par l'entreprise sur le long terme. Le CRT collabore étroitement avec des institutions universitaires, scientifiques et autres de premier plan à l'échelle mondiale, afin de construire l'avenir de l'aérospatiale.

Les domaines techniques traités au sein du CRT sont multiples. On peut citer :

- La recherche de technologies à très fort potentiel telles que les énergies du futur.
- La recherche dans les nouvelles technologies de communication.
- Le développement d'algorithmes d'Intelligence artificielle et de big Data favorisant la prise de décision.
- L'étude et le développement de matériaux et solutions de processus durables.
- La recherche de nouvelles technologies d'électrification pour un futur plus électrique.

Mon stage s'est déroulé au sein de l'équipe **VPE** (***Virtual Product Engineering***) à Issy-les-Moulineaux, membre du CRT, et dont la mission est de faire progresser les technologies de modélisation, de simulation et d'intégration numériques.

Chapitre 2

Introduction générale

2.1 Contexte

Les équations aux dérivées partielles, en abrégé EDPs, permettent de modéliser divers phénomènes du monde réel. Celles-ci trouvent leur application dans de nombreux domaines appliqués de l'ingénierie et de la physique. On les retrouve aussi bien en Mécanique des fluides et en électromagnétisme, qu'en dynamique des structures ou encore en Mathématiques Financières. L'un des grands domaines d'application des EDPs est la simulation aéronautique, où diverses équations telles que les équations de Navier-Stokes, d'Euler ou encore d'Helmholtz, permettent de modéliser divers phénomènes physiques.

2.2 Problématique

Dans le cadre Airbus, et aéronautique en général, l'une des thématiques les plus importantes est celle de la pollution sonore. Face à cette problématique, nous avons **les équations de Goldstein**, permettant de modéliser et de simuler les radiations acoustiques émises par des aéronefs. Résoudre ces équations permettrait d'optimiser le design de ces derniers, les formes géométriques d'un aéronef étant créatrices de sources de bruit. Les équations de Goldstein s'écrivent de la manière suivante (en 2D) :

Trouver $\varphi \in H^1(\Omega)$, $\xi \in H(\Omega, \mathbf{M}_0) := \{\xi \in L^2(\Omega)^d / \mathbf{M}_0 \cdot \nabla \xi \in L^2(\Omega)^d\}$
tel que :

$$\begin{cases} (-ik_0 + \mathbf{M}_0 \cdot \nabla)^2 \varphi - \Delta \varphi - \nabla \cdot \xi = f & \text{sur } \Omega, \\ (-ik_0 + \mathbf{M}_0 \cdot \nabla) \xi + (\xi \cdot \nabla) \mathbf{M}_0 + \omega_0 \times \nabla \varphi = \mathbf{0} & \text{sur } \Omega. \end{cases}$$

où ω_0 est la vorticit  de l' coulement : $\omega_0 := \nabla \times \mathbf{M}_0$.

Avec conditions aux limites :

$$\begin{cases} \varphi = 0 & \text{sur } \partial\Omega, \\ \xi = \mathbf{0} & \text{sur } \partial\Omega^- := \{x \in \partial\Omega / \mathbf{M}_0 \cdot \mathbf{n}(x) < 0\}, \end{cases}$$

— Param tres physiques :

- Le nombre d'onde : k_0 (pouvant $\in \mathbb{C}$),
- La vitesse de l' coulement (en Mach) : $\mathbf{M}_0 : \mathbb{R}^d \rightarrow \mathbb{R}^d$,

- La source acoustique $f : \varphi : \Omega \rightarrow \mathbb{C}$.

— Inconnues

- L'inconnue acoustique : $\varphi : \Omega \rightarrow \mathbb{C}$,
- L'inconnue hydrodynamique : $\xi : \Omega \rightarrow \mathbb{C}^d$.

La **quantit  d'int r t pour Airbus**  tant la **pression acoustique** :

$$p = -(-ik_0 + \mathbf{M}_0 \cdot \nabla) \varphi.$$

Ce syst me d' quations peut  tre r  crit sous la forme suivante :

$$\begin{pmatrix} \mathcal{H}_{[k_0, \mathbf{M}_0]} & \mathcal{D} \\ \mathcal{V}_{[\mathbf{M}_0]} & \mathcal{T}_{[k_0, \mathbf{M}_0]} \end{pmatrix} \begin{pmatrix} \varphi \\ \xi \end{pmatrix} = \begin{pmatrix} f \\ \mathbf{0} \end{pmatrix}. \quad (2.1)$$

— Op rateur d'Helmholtz convect  : $\mathcal{H}_{[k_0, \mathbf{M}_0]}(\varphi) := (-ik_0 + \mathbf{M}_0 \cdot \nabla)^2 \varphi - \Delta \varphi$

— Op rateur de transport :

$$\mathcal{T}_{[k_0, \mathbf{M}_0]}(\xi) := (-ik_0 + \mathbf{M}_0 \cdot \nabla) \xi + (\xi \cdot \nabla) \mathbf{M}_0 \quad (2.2)$$

— Op rateurs de couplage $\mathcal{D}(\xi) := -\nabla \cdot \xi$ and $\mathcal{V}_{[\mathbf{M}_0]}(\varphi) := \omega_0 \times \nabla \varphi = (\nabla \times \mathbf{M}_0) \times \nabla \varphi$

Les  quations de Goldstein sont cependant tr s co teuses   r soudre en utilisant des *solveurs* classiques d'EDPs, ceci d    l'inverse de l'op rateur de transport.

2.3 Objectifs

Pour cette raison, on se tourne vers des m thodes d'apprentissage profond qui ont connu un franc succ s dans le domaine des EDPs, notamment

avec l'introduction des PINNs, ou encore des méthodes basées sur l'apprentissage direct d'un opérateur entre espaces fonctionnels. Nous nous intéressons ici à ce deuxième type de méthodes, afin de résoudre des versions simplifiées de l'équation correspondante à l'opérateur [2.2](#) en 1D et en 2D.

Chapitre 3

Machine Learning pour les EDPs

Les équations aux dérivées partielles (EDPs) sont fondamentales pour la compréhension de nombreux phénomènes physiques. En général, ces équations sont résolues à l'aide de méthodes numériques telles que les éléments finis, les différences finies ou les volumes finis et s'appuient sur une discrétisation du domaine physique. Ces méthodes ont l'avantage de donner des résultats très précis et pertinents. Cependant, dans de nombreux cas concrets où l'EDP est assez complexe, de très fines discrétisations sont nécessaires pour atteindre la précision souhaitée : la durée d'une résolution de l'équation devient alors beaucoup trop longue pour pouvoir lancer tous les calculs dont les ingénieurs auraient besoin. La popularité des méthodes d'apprentissage automatique a développé un fort intérêt dans l'application de cette science à la résolution d'équations aux dérivées partielles, notamment à travers l'utilisation des réseaux de neurones. On peut distinguer deux grands groupes de méthodes utilisant des techniques d'apprentissage automatique afin de résoudre des EDPs.

La première méthode, introduite dans [2], puis grandement approfondie par [15] consiste à paramétrer la solution de l'EDP, pour une instance de paramètres donnée, par un réseau de neurones $NN : D \times \Theta \rightarrow \mathbb{R}$, D étant le domaine physique de calcul et Θ l'espace des paramètres du réseau. L'avantage de ce type de méthodes est que par définition, elles ne dépendent pas de la discrétisation faite. Cependant, elles présentent deux inconvénients majeurs à savoir le besoin d'entraîner le réseau pour chaque nouvelle instance de paramètres, ce qui est très coûteux, et la nécessité de pouvoir écrire l'équation de façon concrète.

La deuxième méthode consiste à apprendre directement l'opérateur solution, afin d'éviter d'entraîner le réseau à chaque fois comme précédemment. Basé sur le succès des réseaux de neurones convolutionnels (CNNs), diffé-

rents travaux se sont focalisés sur l'apprentissage de cet opérateur au travers d'un CNN profond $NN : \mathbb{R}^K \times \Theta \rightarrow \mathbb{R}^K$, le domaine D étant ici discrétisé en K points. Ce type de méthodes s'avère cependant dépendant de la discrétisation choisie, signifiant une modification de l'architecture du réseau à chaque fois qu'un maillage différent est fait.

Récemment, de nouvelles recherches ont été faites afin de proposer différentes méthodes d'apprentissage par des réseaux de neurones, étant indépendantes de la discrétisation choisie, et permettant d'apprendre des opérateurs entre espaces de dimensions infinies. Ceci s'est fait avec l'introduction du Deeponet [12], suivi d'architectures de plus en plus élaborées [1], [13], [14], [10], [9], [8], [11]. Notons enfin que l'idée de généraliser les réseaux de neurones aux espaces de dimensions infinies a longuement été étudiée depuis les années 90 [16], [6], [4], mais n'a connu de réel succès que récemment dû à l'utilisation massive d'architectures telles que les CNNs, les RNNs, etc ...

Chapitre 4

Le problème d'apprentissage

Dans le cadre de ce stage, nous nous sommes intéressés aux différents cas suivants, progressant continuellement en difficulté. Ces équations correspondent à des versions simplifiées de l'opérateur de transport 2.2 que nous cherchons à inverser.

4.1 Equation de transport : Cas 1D

4.1.1 Cas m_0 constant

Le premier cas que nous considérons est le cas où l'écoulement m_0 est constant, dérivé du cas non constant ci-dessous. Le problème, étant donné $g : [0, 1] \rightarrow \mathbb{C}$, est de trouver $\psi : [0, 1] \rightarrow \mathbb{C}$ vérifiant :

$$\begin{cases} -ik_0\psi(x) + m_0\psi'(x) = g(x), & \text{for } x \in [0, 1], \\ \psi(0) = 0 \end{cases}$$

avec $m_0 > 0$.

La solution analytique s'écrit :

$$\psi(x) = \mathcal{T}_{[k_0, m_0]}^{-1} g(x) = \frac{1}{m_0} \int_0^x \exp\left(i \frac{k_0}{m_0} (x-s)\right) g(s) ds$$

4.1.2 Cas m_0 non constant

Le second cas de figure est la même équation de transport :

$$\begin{cases} -ik_0\psi(x) + m_0(x)\psi'(x) = g(x), & \text{for } x \in [0, 1], \\ \psi(0) = 0, \end{cases}$$

avec : $m_0 : [0, 1] \rightarrow \mathbb{R}_+^*$.

La solution analytique s'écrit ici :

$$\psi(x) = \mathcal{T}_{[k_0, m_0]}^{-1} g(x) = \int_0^x \exp\left(\int_s^x i \frac{k_0}{m_0(\tau)} d\tau\right) \frac{g(s)}{m_0(s)} ds.$$

On retrouve la même égalité ci-dessus pour m_0 constant.

4.2 Equation de transport : Cas 2D

On considère ensuite le cas 2D, prenant en compte un coefficient d'écoulement non constant. L'équation 2D scalaire est considéré dans un "shear flow" (signifiant que $M_0 = M_0(y)e_x$) et s'écrit :

$$\begin{cases} M_0(y) \partial_x \psi(x, y) - ik_0 \psi(x, y) = g(x, y) M'_0(y), & (x, y) \in]x_1, x_2[\times]y_1, y_2[, \\ \psi(x_1, y) = 0, & \forall y \in]y_1, y_2[. \end{cases}$$

La solution (inverse de l'opérateur de transport) s'écrit :

$$\psi(x, y) = \left(\mathcal{T}_{[k_0, M_0]}^{-1} g \right)(x, y) = \frac{M'_0(y)}{M_0(y)} \int_{x_1}^x e^{\frac{ik_0}{M_0(y)}(x-s)} g(s, y) ds.$$

4.3 Bases de données

Pour notre étude, nous devons premièrement générer des bases de données. Le domaine D est à chaque fois **discretisé en K points** et on considère $\Gamma = \{x_1, \dots, x_K\}_{k=1}^K \subset D$ l'ensemble des points de la discrétisation de D , avec $x_1 = 0 < x_2 < \dots < x_K = 1$ une subdivision de $[0, 1]$. Par souci de clarté, on notera la vitesse de l'écoulement en 1D(m_0) par m dans la suite.

On prendra comme **paramètres d'entrée** : $a_i = (x, m_i, Re(g)_i, Im(g)_i)$, avec $x = (x_k)_{1 \leq k \leq K}$, $Re(g)_i = [Re(g(x_k))]_{1 \leq k \leq K}$, $Im(g)_i = [Im(g(x_k))]_{1 \leq k \leq K}$, $i = 1, \dots, N$ avec :

- m étant la vitesse de l'écoulement
- $Re(g)$ la partie réelle de la source g
- $Im(g)$ la partie imaginaire de la source g
- N le nombre d'observations

et comme **paramètres de sortie** ($Re(\psi)_i, Im(\psi)_i$)

avec : $Re(\psi)_i = [Re(\psi(x_k))]_{1 \leq k \leq K}$, $Im(\psi)_i = [Im(\psi(x_k))]_{1 \leq k \leq K}$, $i = 1, \dots, N$ où

- $Re(\psi)$ la partie réelle de la solution ψ

— $Im(\psi)$ la partie imaginaire de la solution ψ

Soit $\mathcal{P}_L$ une méthode d'échantillonnage telle que :

$$a \sim \mathcal{P}_L([a_{min}, a_{max}]) \Leftrightarrow a = \sum_{l=1}^L \alpha_l T_l, \quad \alpha_1, \dots, \alpha_n \stackrel{i.i.d}{\sim} \mathcal{U}([0, 1])$$

où les T_l sont des polynômes de Chebyshev normalisés tels que $a \in [a_{min}, a_{max}]$

Cas 1D, m non constant : On génère des valeurs de m et g de la façon suivante :

- $\mathbf{m} = \{m_k\}_{k=1}^K = \{m(x_k)\}_{k=1}^K$ avec $m \sim \mathcal{P}_{10}(m_{range})$
- $\mathbf{g} = \{g_k\}_{k=1}^K = \{g(x_k)\}_{k=1}^K$ avec $g(x_k) = a(x_k)e^{i \sum_{l=1}^{k-1} \alpha(x_k)(x_{l+1}-x_l)}$ où $\alpha \sim \mathcal{P}_{10}(\alpha_{range})$, $a \sim \mathcal{P}_{10}([-1, 1])$, $k = 1, \dots, K$
- Les $\{\psi_k\}_{k=1}^K$ sont simulés en à l'aide d'un *solver* s'appuyant sur la solution exacte.

En 1D, on considère les deux bases de données suivantes pour le cas où m n'est pas constant :

- Base de données **db1**
 - Nombre de points du maillage (discrétisation spatiale) : $K = 512$
 - Intervalle de valeurs de m : $\mathbf{m} \in \mathbf{m}_{range} = [0.4, 0.5]$
 - Intervalle de valeurs de α : $\alpha \in \alpha_{range} = [40, 50]$
 - $k_0 = 90$
- Base de données **db2**
 - Nombre de points du maillage (discrétisation spatiale) : $K = 512$
 - Intervalle de valeurs de m : $\mathbf{m} \in \mathbf{m}_{range} = [0.2, 0.7]$
 - Intervalle de valeurs de α : $\alpha \in \alpha_{range} = [25, 150]$
 - $k_0 = 90$

Note 1 : La nécessité d'étudier deux bases de données avec différentes plages de variation des paramètres est justifiée par le fait qu'en situation réelle, nous ne pouvons pas savoir dans quels intervalles les données seront. Il est donc utile de faire une étude sur différents cas de figure.

Note 2 : Nous ne développerons pas le cas où m est constant ici, car toutes les discussions faites sur **db1** et **db2** dans la suite restent vraies pour ce cas de figure.

Les figures 4.1 et 4.2 (respectivement) illustrent quelques observations générées à partir des bases de données db1 et db2 (respectivement).

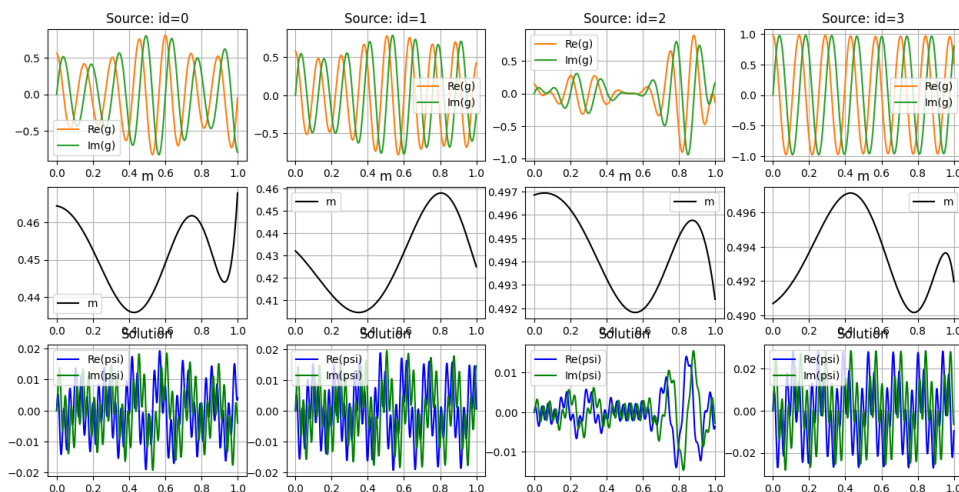


FIGURE 4.1 – Observations tirées de db1 : La longueur d'onde étant $\lambda = \frac{m}{k}$, le nombre d'oscillations est inversement proportionnel à la valeur de m et donc, plus m est grand, moins on a d'oscillations.

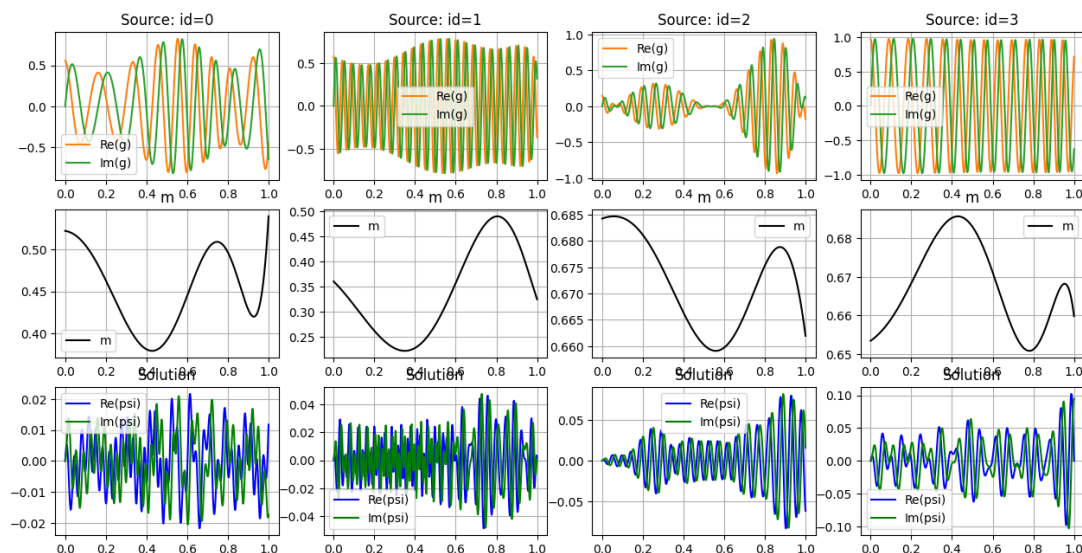


FIGURE 4.2 – Observations tirées de db2

Cas 2D :

- Les valeurs de M_0 sont générées en utilisant la formule suivante :

$$M_{0k} = M_0(y_k) = m1 + (m2 - m1)e^{-\frac{y_k - y_m}{2\sigma^2}}$$

avec $m1 \sim \mathcal{P}_{10}(m_{range1})$, $m2 \sim \mathcal{P}_{10}(m_{range2})$, $\sigma \sim \mathcal{P}_{10}(\sigma_{range})$

- Les valeurs de g sont générées en utilisant la formule suivante :

$$g_{k,k'} = g(x_k, y_{k'}) = a(x_k)e^{i\alpha_x(x_k)x_k}e^{i\alpha_y(y_{k'})y_{k'}}$$

avec $a \sim \mathcal{P}_{10}(a_{range})$, $\alpha_x \in \mathcal{P}_{10}(\alpha_{xrange})$, $\alpha_y \in \mathcal{P}_{10}(\alpha_{yrange})$, $(x_k, y_{k'}) \in \Gamma \times \Gamma$

Base de données db3 :

- Nombre de points du maillage (discrétisation spatiale) : $K = 256 \times 256$
- Intervalle de valeurs de $m1$: $m1 \in m_{range1} = [0.2, 0.3]$
- Intervalle de valeurs de $m2$: $m2 \in m_{range2} = [0.4, 0.5]$
- Intervalle de valeurs de σ : $\sigma \in \sigma_{range} = [0.02, 0.2]$
- $y_m = 0.1$
- Intervalle de valeurs de a : $a \in a_{range} = [1, 5]$
- Intervalle de valeurs de α_x : $\alpha_x \in \alpha_{xrange} = [1, 20]$
- Intervalle de valeurs de α_y : $\alpha_y \in \alpha_{yrange} = [1, 20]$
- $k_0 = 10$

La figure 4.3 illustre quelques observations générées à partir de la base de données db3.

4.4 Fonction de perte

Le choix de la fonction à minimiser est crucial en apprentissage. Nous considérerons ici l'**erreur L2 relative**, qui est l'une des plus utilisées dans la résolution d'EDPs par apprentissage, dû à ses bonnes propriétés. Elle est définie comme suit :

Soit D le domaine de l'EDP et $\Gamma = \{x_1, \dots, x_K\}$ l'ensemble des points de discrétisation de D .

Soit ψ et ψ' avec $\psi_k, \psi' \in \mathbb{C}$.

On discrétise ψ et ψ' sur Γ et on note $\psi = (\psi_k)_{k=1,\dots,K} = (\psi(x_k))_{k=1,\dots,K}$ et

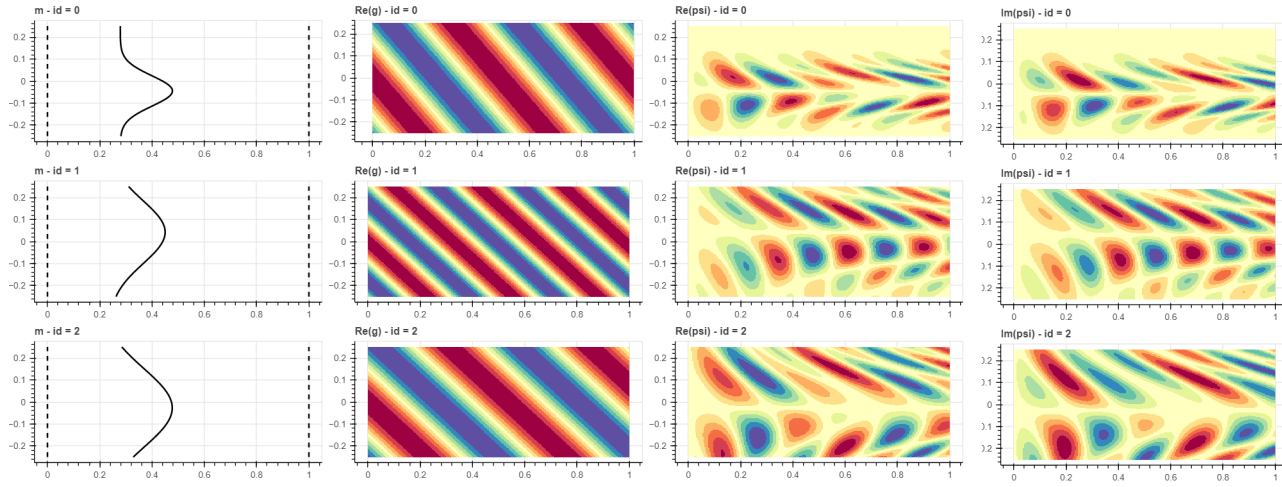


FIGURE 4.3 – Observations tirées de db3 : Comme on peut le voir sur les figures de la première colonne, l'écoulement est pris ici comme étant unidirectionnel (constant à y fixé)

$\psi' = (\psi'_k)_{k=1,\dots,K} = (\psi'(x_k))_{k=1,\dots,K}$ On définit :

$$\begin{aligned} loss_{data}(\psi, \psi') &= \frac{\|\psi - \psi'\|_{L^2(\Gamma)}}{\|\psi\|_{L^2(\Gamma)}} \\ &= \sqrt{\frac{\sum_{k=1}^K |Re(\psi_k) - Re(\psi'_k)|^2 + |Im(\psi_k) - Im(\psi'_k)|^2}{\sum_{k=1}^K |Re(\psi_k)|^2 + |Im(\psi_k)|^2}} \end{aligned}$$

La fonction de perte à minimiser s'écrit donc ici :

$$\mathcal{L}_{data} = \mathbb{E}[loss_{data}(\cdot)].$$

Chapitre 5

Travail réalisé : Les opérateurs neuronaux (NO)

Les opérateurs neuronaux constituent une généralisation des réseaux de neurones, permettant d'apprendre des opérateurs entre espaces fonctionnels à partir d'observations (entrées-sorties). Tout leur intérêt réside dans le fait qu'ils permettent d'apprendre directement toute une famille d'EDPs, sans besoin d'entraînement du réseau pour chaque nouvelle instance de paramètres.

Notre problème peut s'écrire de la façon suivante :

Soit $D \subset \mathbb{R}^d$ le domaine de l'EDP.

On considère l'opérateur $G : F \rightarrow U$ avec $F = F_m(D, \mathbb{R}) \times F_g(D, \mathbb{C})$ et $U = U_\psi(D, \mathbb{C})$ où F_m, F_g et U_ψ des espaces de Banach de fonctions.

Supposons que pour nos observations $\{a, \psi\} = \{a_i, \psi_i\}_{i=1}^N$ où $a_i = (m_i, \text{Re}(g)_i, \text{Im}(g)_i)$, on ait $\psi_i = G(a_i)$.

Notre but est d'approximer G par une fonction $G_\theta : F_h \times \Theta \rightarrow U_h$ pour un espace de paramètres Θ donné, tel que $G_\theta \approx G$ (formulation du problème d'apprentissage en dimension infini).

Nous avons donc un problème classique d'optimisation en apprentissage automatique :

$$\underset{\theta \in \Theta}{\operatorname{argmin}} \mathbb{E}[\text{loss}_{\text{data}}(G(a), G_\theta(a))] \approx \underset{\theta \in \Theta}{\operatorname{argmin}} \left[\frac{1}{N} \sum_{j=1}^N \text{loss}_{\text{data}}(G(a), G_\theta(a)) \right]$$

Par souci de simplicité, considérons l'écriture de la solution ψ en 1D (ce qui suit restant vrai pour le cas 2D) :

$$\begin{aligned}\psi(x) &= \mathcal{T}_{[k_0, m_0]}^{-1} g(x) \\ &= \int_0^x \exp\left(\int_s^x i \frac{k_0}{m_0(\tau)} d\tau\right) \frac{g(s)}{m_0(s)} ds.\end{aligned}$$

Ce qui donne :

$$\psi(x) = \int_D G(x, s) g(s) ds \quad (5.1)$$

avec

$$G(x, s) = \mathbb{1}_{x-s \geq 0} \exp\left(\int_s^x i \frac{k_0}{m_0(\tau)} d\tau\right) \frac{1}{m_0(s)}, D = [0, 1].$$

La fonction G étant continue, on peut la paramétriser par un réseau de neurones qu'on appellera κ_ϕ (Théorème d'approximation universelle).

Suivant [10], (5.1) conduit à l'architecture itérative suivante, qu'on considère comme étant la généralisation des réseaux de neurones aux espaces de dimensions infinis.

Pour $t = 0, \dots, T-1$, T étant le nombre de couches du réseau,

$$v_{t+1}(x) = \sigma[Wv_t(x) + \int_D \kappa_\phi(x, s, m(x), m(s)) v_t(s) \nu_x(ds)]$$

où les $v_i, i = 0, \dots, T-1$ sont des fonctions à valeurs dans $\mathbb{R}^{d_v}$

avec $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ une fonction d'activation, $W \in \mathbb{R}^{d_v}$ un vecteur de poids synaptiques, ν_x une mesure de Borel qu'on prendra ici comme étant la mesure de Lebesgue, et $\kappa_\phi : \mathbb{R}^{2(d+1)} \rightarrow \mathbb{R}^{d_v \times d_v}$.

Plus clairement, l'architecture que nous considérons en pratique est la suivante :

$$\begin{cases} v_0(x) = P(x, m(x), \text{Re}(g)(x), \text{Im}(g)(x)) + p \\ v_{t+1}(x) = \sigma(Wv_t(x) + \mathcal{K}(m; \phi)v_t(x)) \\ \psi_{NN}(x) = Qv_T(x) + q \end{cases}$$

avec $P \in \mathbb{R}^{d_v \times 4}$ une transformation linéaire et locale en grande dimension des données d'entrée, permettant d'augmenter leur expressivité, $p \in \mathbb{R}^{d_v}$ un vecteur de biais, $Q \in \mathbb{R}^{2 \times d_v}$ une matrice permettant de projeter la sortie de la dernière couche du réseau dans l'espace adéquat (ici en dimension 2, puisqu'on a deux paramètres de sortie), $q \in \mathbb{R}^2$ un vecteur de biais et $\mathcal{K}(m; \phi)v_t(x) = \int_D \kappa_\phi(m(x), m(y), x, y) v_t(y) dy$

5.1 Opérateur neuronal de Fourier (FNO)

5.1.1 Présentation

Introduit dans [8], l'opérateur neuronal de Fourier est une architecture rapide et efficace, basée sur la paramétrisation du réseau κ_ϕ en Fourier.

Soit $\kappa_\phi : \mathbb{R}^{2(d+1)} \rightarrow \mathbb{R}^{d_v \times d_v}$, le réseau de neurones introduit ci-dessus.

En supposant que :

- κ_ϕ ne dépend pas de m ,
- $\kappa_\phi(x, y) = \kappa_\phi(x - y)$.

On obtient :

$$\begin{aligned} \mathcal{K}(\phi)v_t(x) &= \int_D \kappa_\phi(x, y)v_t(y)dy \\ &= \int_D \kappa_\phi(x - y)v_t(y)dy \\ &= \kappa_\phi \star v_t(x). \end{aligned}$$

Soit $\mathcal{F}$ la transformée de fourier. On a :

$$\begin{aligned} \mathcal{K}(\phi)v_t(x) &= \mathcal{F}^{-1}(\mathcal{F}(\kappa_\phi \star v_t(x))) \\ &= \mathcal{F}^{-1}(\mathcal{F}(\kappa_\phi) \cdot \mathcal{F}(v_t))(x) \text{ avec } \mathcal{F}(\kappa_\phi) \in \mathbb{C}^{d_v \times d_v} \text{ et } \mathcal{F}(v_t) \in \mathbb{C}^{d_v}. \end{aligned}$$

Le domaine D étant discrétisé, on utilise ici la **Transformée de Fourier Discrète**. En pratique, après avoir calculé la transformée de Fourier de $\mathcal{F}(\kappa_\phi)$, on ne retient que les k_{max} premiers modes de Fourier, ce qui est indispensable pour avoir une architecture rapide. On multipliera donc un tenseur $\mathcal{F}(\kappa_\phi) \in \mathbb{C}^{k_{max} \times d_v \times d_v}$ avec un tenseur $\mathcal{F}(v_t) \in \mathbb{C}^{k_{max} \times d_v}$

$$\mathcal{F}((\kappa_\phi) \cdot \mathcal{F}(v_t))_{i,j} = \sum_{k=1}^{d_v} \mathcal{F}(\kappa_\phi)_{i,j,k} \cdot \mathcal{F}(v_t)_{i,k}$$

avec $i = 1, \dots, k_{max}$ et $j = 1, \dots, d_v$

Remarque : En pratique, $\mathcal{F}(\kappa_\phi)$ est directement paramétré en Fourier (en utilisant la classe **Parameter** de Pytorch), ce qui justifie qu'on enlève la dépendance de κ_ϕ en m .

La figure 5.1 et l'algorithme 1 ci-dessous résument la résolution de l'équation de transport en 1D en utilisant l'opérateur neuronal de Fourier, algorithme qui peut être généralisé au cas 2D.

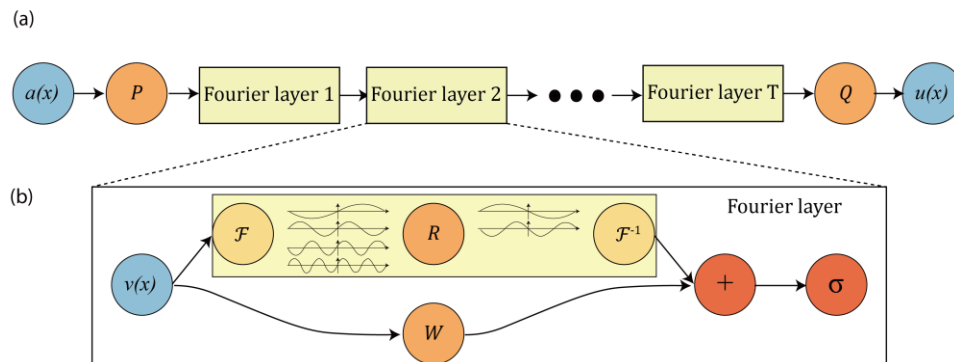


FIGURE 5.1 – **Architecture du FNO.**(a) : Partant de nos données d'entrée, la première couche du réseau (**P**) consiste à passer en grande dimension par une transformation locale des données, augmentant ainsi leur expressivité. Ensuite, on applique un nombre T de *couches de Fourier*, et enfin on projette par **Q** la sortie de la dernière couche de Fourier dans la dimension de sortie souhaitée . (b) : On détaille ici les opérations qui sont faites dans une couche de Fourier. Après le passage en grande dimension par **P**, d'une part on applique une Transformée de Fourier Rapide (**FFT**) au tenseur, puis on ne garde que les k_{max} premiers modes de Fourier, qu'on multiplie avec un tenseur complexe R , et enfin on fait le Fourier inverse du tenseur obtenu pour revenir dans le domaine spatial, et d'autre part on applique à ce même tenseur initial une transformation (couche convolutionnelle ou couche dense si on est en dimension 1) et on additionne les deux sorties qu'on compose par la suite avec une non-linéarité.

Algorithm 1 FNO : Equation de transport 1D**Données d'entrée** : $\{x, m_j, Re(g)_j, Im(g)_j\}_{j=1}^N$ **Données de sortie** : $\{Re(\psi_j), Im(\psi_j)\}_{j=1}^N$ **Résultat** : Opérateur $G_\theta : F \times \Theta \rightarrow U$

- 1: Données d'entrée $a = \{a_j\}_{j=1}^{batch} = \{x, m_j, Re(g)_j, Im(g)_j\}_{j=1}^{batch}$ ▷
 $[batch, K, 4]$
- 2: On applique P pour obtenir $v_0 : v_0 = P(a)$ ▷ $[batch, K, d_v]$
- 3: Permutation d'indexes : $v_0 = permute(v_0)$ ▷ $[batch, d_v, K]$
- 4: Pour $i = 1, \dots, T$,
 - Couches de Fourier (d'une part)
 - Paramétrisation de $\mathcal{F}(\kappa_\phi)$ ▷ $[d_v, d_v, k_{max}]$
 - FFT de $v_{i-1} : \mathcal{F}(v_{i-1})$ ▷ $[batch, d_v, k_{max}]$
 - Inverse FFT du produit : $sortie = IFFT(\mathcal{F}(\kappa_\phi) \cdot \mathcal{F}(v_{i-1}))$ ▷
 $[batch, d_v, K]$
 - Convolution (ou couche Dense en 1D) d'autre part
- 5: $sortie = \sigma(sortie)$ ▷ $[batch, d_v, K]$
- 6: Permutation d'indexes : $sortie = permute(sortie)$ ▷ $[batch, K, d_v]$
- 7: On applique Q : $\psi_{predict} = Q(sortie)$ ▷ $[batch, K, 2]$

5.1.2 Résultats**Cas 1D :**

Le tableau 5.1 donne les résultats obtenus par le FNO en dimension 1. Pour db1 et db2, nous construisons le modèle à partir des paramètres suivant :

Paramètres :

- $n_{train} = 10.000$
- $n_{test} = 2.000$
- $T = 4$
- $d_v = 64$
- $k_{max} = 64$
- $\sigma = GELU$
- $epochs = 1500$
- $lr = 0.001$

TABLE 5.1 – Résultats FNO 1D

Modèles	Nbre de paramètres	loss_train	loss_test
FNO 1D (db1)	1,074,114	0.00467	0.00702
FNO 1D (db2)	1,074,114	0.01251	0.54834

Comme on peut le constater, l'apprentissage de l'opérateur est beaucoup plus compliqué pour **db2** que pour **db1**, dû au fait que les plages de variations des paramètres dans le cas de db2 sont plus grandes que celles de db1. On observe (figure 5.3) ici un phénomène d'**overfitting** du modèle db2. Les deux techniques naturelles pour combattre ce phénomène sont l'augmentation des données d'entraînement et la régularisation. Cependant, régulariser le modèle ici ne permettrait qu'une très faible amélioration, et deviendrait obsolète pour une autre base de données avec différentes variations des paramètres.

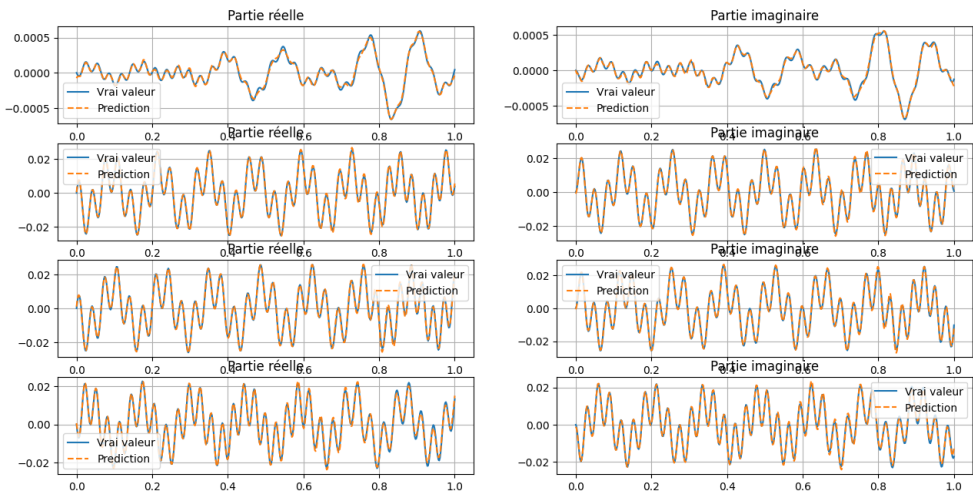


FIGURE 5.2 – Résultats FNO 1D - Cas de db1 : On affiche ici les 4 (sur les 2000 de test) moins bon résultats obtenus.

Cas 2D :

Le tableau 5.2 donne les résultats obtenus par le FNO en dimension 1. Nous construisons le modèle à partir des paramètres suivants :

Paramètres :

→ $n_{train} = 3000$

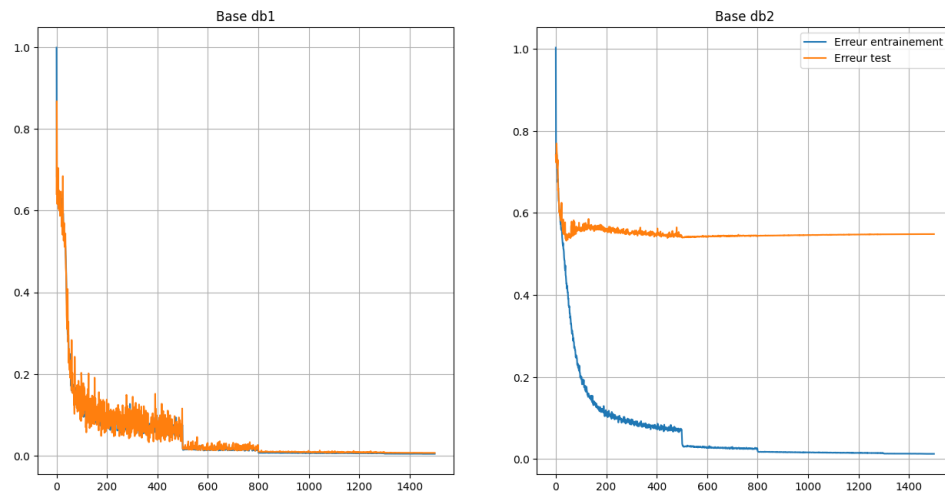


FIGURE 5.3 – **Minimisation de la fonction de perte L2 relative - 1D** : Les "points de chute" observés sur les courbes sont dû au fait que nous diminuons la valeur du *learning rate* tous les 200 epochs

- $n_{test} = 100$
- $T = 4$
- $d_v = 64$
- $k_{max} = 64$
- $\sigma = GELU$
- $epochs = 1500$
- $lr = 0.001$

TABLE 5.2 – Résultats FNO 2D

Modèle	Nbre de paramètres	loss_train	loss_test
FNO 2D	2,368,194	0.019	0.03

Note : Les différents modèles ont été entraînés ici sur un NVIDIA RTX3060 (1 GPU). En 1D, la durée moyenne (avec les paramètres définis ci-dessus) de $1.05sec/epochs$ et en 2D, elle est de $59.5sec/epochs$.

Résultats additionnels

Le FNO ne dépendant pas du nombre de points de la discrétisation, on peut se demander comment un modèle entraîné sur une certaine résolution, performe sur une autre. Dans le tableau 5.3, on teste notre modèle 1D

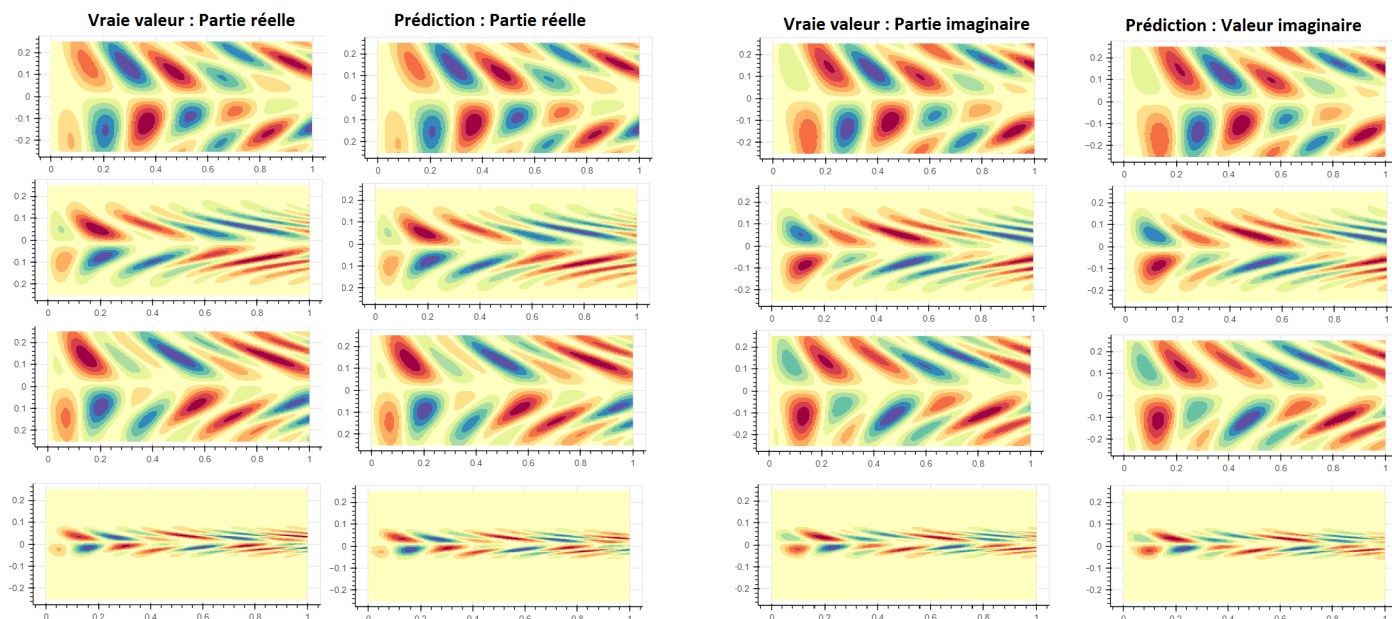


FIGURE 5.4 – Résultats FNO 2D

(db1) entraîné ci-dessus, sur des discrétisations différentes. On constate que le modèle prédit mieux lorsque le nombre de points de la discrétisation du test est plus grand que celui des données d'entraînement ($K = 512$). Dans le tableau 5.4, on entraîne et teste le modèle 1D avec les mêmes paramètres que précédemment mais sur différentes résolutions, afin de vérifier la consistance de l'erreur.

TABLE 5.3 – Résultats FNO 1D, différentes résolutions : On teste le modèle entraîné ci-dessus avec $K = 512$ sur différentes résolutions

Modèle	K	loss_test
FNO 1D (db1)	256	0.096
FNO 1D (db1)	1024	0.0465
FNO 1D (db1)	2048	0.0746

TABLE 5.4 – Résultats FNO 1D : Consistance de l'erreur(apprentissage sur d'autres résolutions)

Modèle	K (train)	K (test)	loss_train	loss_test
FNO 1D	256	256	0.007	0.0083
FNO 1D	1024	1024	0.0056	0.009

5.1.3 Limites du FNO

L'algorithme FNO, bien que donnant de bons résultats dans nos cas de figure comme illustré ci-dessus, présente quelques inconvénients.

Premièrement, dû à l'utilisation de la Transformée de Fourier Rapide (FFT), il est indispensable d'avoir une discrétisation **uniforme** du domaine D . Or, dans de nombreux cas pratique, certaines zones du domaine peuvent nécessiter une discrétisation plus fine que d'autres, ce qui impliquerait d'avoir un maillage non uniforme.

De plus, l'algorithme FNO étant une méthode complètement *data-driven*, il est nécessaire d'avoir une quantité de données importante, afin d'apprendre l'opérateur. Cependant, recueillir des données en pratique n'est pas toujours chose aisée. Nous explorons donc les méthodes suivantes, afin de voir dans quelle mesure il est possible de surmonter ces obstacles.

5.2 Réseau à noyau graphique (GNO)

5.2.1 Présentation

Reconsidérons l'architecture itérative :

$$\begin{cases} v_0(x) = P(x, m(x), \text{Re}(g)(x), \text{Im}(g)(x)) + p, \\ v_{t+1}(x) = \sigma(Wv_t(x) + \int_D \kappa_\phi(x, s, m(x), m(s))v_t(s)v_x(ds)) \quad t = 0, \dots, T-1, \\ \psi_{NN}(x) = Qv_T(x) + q. \end{cases}$$

Supposons $v_x(ds) = \mathbb{1}_{B(x,r)}ds$, signifiant qu'on choisit ici la mesure de Borel comme étant la mesure de Lebesgue sur la boule de centre x et de rayon r . On obtient :

$$v_{t+1}(x) = \sigma(Wv_t(x) + \int_{B(x,r)} \kappa_\phi(x, s, m(x), m(s))v_t(s)ds). \quad (5.2)$$

Dans [10], l'intégrale présente dans l'expression (5.1) est approchée par Monte Carlo, en utilisant la structure de **message passing** énoncée dans [3], nous permettant d'écrire (5.1) sous la forme suivante :

$$v_{t+1}(x) = \sigma(Wv_t(x) + \frac{1}{\text{card}(N(x))} \sum_{y \in N(x)} \kappa(e(x, y))v_t(y)), \quad (5.3)$$

avec $N(x)$ l'ensemble des voisins de x (dans ce cas ci, il s'agit des $y \in B(x, r)$), $W \in \mathbb{R}^{n \times n}$, $e(x, y) = (x, y, m(x), m(y))$ appelé *edges features*, et

$\kappa(e(x, y))$ un réseau de neurones prenant $e(x, y)$ en entrée et en sortie une matrice de taille $d_v \times d_v$.

L'architecture de *message passing* [3] permet d'aggréger l'information présente chez les voisins de chaque node x . Le domaine étant discrétisé, les nodes $\{x_j\}_{j=1}^K$ sont choisis comme étant les différents points de discrétisation. L'avantage du GNO est que, contrairement au FNO, aucune supposition n'a besoin d'être faite sur la structure du maillage. Par construction, un réseau à noyau graphique devrait être capable d'accepter aussi bien un maillage uniforme, qu'un maillage non uniforme. Cependant, le nombre d'*edges* pouvant être de l'ordre de $O(K^2)$, d'importants problèmes de mémoire peuvent assez rapidement survenir en fonction du rayon r et de la discrétisation K choisie.

5.2.2 Résultats

Le tableau 5.5 donne les résultats obtenus en 1D en utilisant un modèle de réseau à noyau graphique. Les paramètres utilisés sont les suivants :

- $n_{train} = 100$
- $n_{test} = 10$
- $K = 256$
- $T = 4$
- $r = 0.03$
- $d_v = 128$
- $lr = 0.001$
- $epochs = 1000$
- $\sigma = ReLU$

κ est un réseau de neurones à propagation avant à 4 couches ($\kappa : (4, d_v, d_v, d_v, d_v^2)$ -*feed forward*), avec comme fonction d'activation une ReLU.

TABLE 5.5 – Résultats GNO 1D

Modèle	loss_train	loss_test
GNO 1D (db1)	0.61	0.65
GNO 1D (db2)	0.7	0.85

5.2.3 Commentaires

Comme nous pouvons observer, le GNO ne donne pas de bons résultats dans la résolution de l'équation de transport en 1D. Différents facteurs peuvent en être la cause :

- Dans le cas 1D, on a :

$$G(x, s) = \mathbb{1}_{x-s \geq 0} \exp\left(\int_s^x i \frac{k_0}{m_0(\tau)} d\tau\right) \frac{1}{m_0(s)}. \quad (5.4)$$

Comme on peut le voir dans l'expression, pour un node x donné, le kernel κ_ϕ devait dépendre de tous les points entre s et x , à savoir $s, s+1, \dots, x$. En restreignant donc la mesure à la boule, on supprime potentiellement de l'information.

- De plus, G étant à valeurs complexes, il est possible que les résultats observés soient liés au fait que κ_ϕ soit à valeurs réelles, causant ainsi une mauvaise approximation de la fonction par le kernel. Dans le cas du FNO, κ_ϕ ayant été paramétré en Fourier, et donc étant à valeurs complexes, il est difficile de faire une analogie entre les deux cas afin d'isoler le problème. Nous avons constaté qu'en pratique, changer les paramètres du réseau κ_ϕ (nombre de couches, nombre de neurones, etc ...) n'améliore pas la qualité des résultats ci-dessus, mais peut dans quelques cas les rendre plus mauvais.

Nous faisons les expériences sur peu de données car le modèle est en général long à entraîner. De plus, nous rencontrons des problèmes d'allocation de mémoire assez rapidement, lorsque par exemple on augmente le rayon r , ou qu'on discrétise les données sur plus de points. Le modèle GNO ne semble donc pour l'instant pas adapté à la résolution de notre équation, et nécessite une étude théorique plus poussée dans ce cas de figure. A cause des observations précédentes, nous n'avons pas appliqué la méthode au cas 2D.

5.3 Opérateur neuronal informé de la physique (PINO)

La troisième et dernière méthode que nous explorons est celle reposant sur une combinaison entre opérateurs neuronaux, [8], [10], [9] et réseaux de neurones informés de la physique [15]. Nous nous intéressons au besoin important de données du FNO, énoncé dans 5.1.3.

5.3.1 Les PINNs

Les réseaux de neurones informés de la physique (*Physics-Informed Neural Networks* ou PINNs), introduits en 2019 par [15], ont pour but de résoudre des EDPs. Contrairement aux réseaux de neurones traditionnels, les PINNs tiennent compte des lois physiques qui régissent l'EDP à résoudre. De même que dans les cas précédents, on discrétise ici le domaine D en

K points et on pose $\Gamma = \{x_1, \dots, x_K\}$. Dans les PINNs, on considère deux fonctions pertes.

- la première fonction de perte, qu'on nomme $loss_{NN}$, est l'erreur entre la valeur réelle et la valeur prédite par un réseau de neurones défini,
- la deuxième fonction de perte, qu'on nommera $loss_{pde}$ est l'erreur entre la valeur prédite et la valeur respectant la physique régit par l'EDP.

La fonction de perte globale considérée est :

$$loss_{total} = loss_{NN} + loss_{pde}.$$

Plus explicitement, considérons notre équation de transport en 1D :

$$\begin{cases} -ik_0\psi(x) + m_0(x)\psi'(x) = g(x), & \text{pour } x \in [0, 1], \\ \psi(0) = 0, \end{cases}$$

avec : $m_0 : [0, 1] \rightarrow \mathbb{R}_+^*$. On aura ici :

- $loss_{NN} = \|\psi - \psi_{pred}\|_{\Gamma}$ où ψ_{pred} est la solution prédite par le réseau de neurones.
- $loss_{pde} = \|ik_0\psi_{pred} + m\psi'_{pred} - g\|_{\Gamma} + |\psi_{pred}(0)|$.

De par l'information physique présente dans la structure des PINNs, les solutions possibles sont restreintes par le réseau, ce qui limite le besoin en données d'apprentissage. Les PINNs présentent cependant un inconvénient majeur, à savoir le besoin d'entraîner un nouveau réseau de neurones pour chaque nouvelle instance de paramètres. La figure 5.5 résume l'architecture des PINNs.

5.3.2 Couplage

La méthode PINO [11] consiste à combiner opérateur neuronal (nous nous intéressons ici au cas du FNO) et PINNs, ceci afin de surmonter les différents obstacles rencontrés par ces derniers. L'idée est de remplacer le réseau de neurones présent dans la structure des PINNs par celui du FNO, ceci afin d'apprendre l'opérateur G_{θ} , ce qui permet d'éviter de faire un entraînement du réseau à chaque nouvelle instance de l'EDP.

La méthode PINO se décompose en deux étapes :

- **Une phase d'entraînement** : Apprentissage de l'opérateur G_{θ} en utilisant les fonctions de perte $loss_{data}$ et $loss_{pde}$. La fonction de perte à minimiser ici s'écrit :

$$loss_{total} = \alpha loss_{data}(\psi, G_{\theta}(a)) + \beta loss_{pde}(G_{\theta}(a), a), \text{ avec } \alpha, \beta > 0.$$

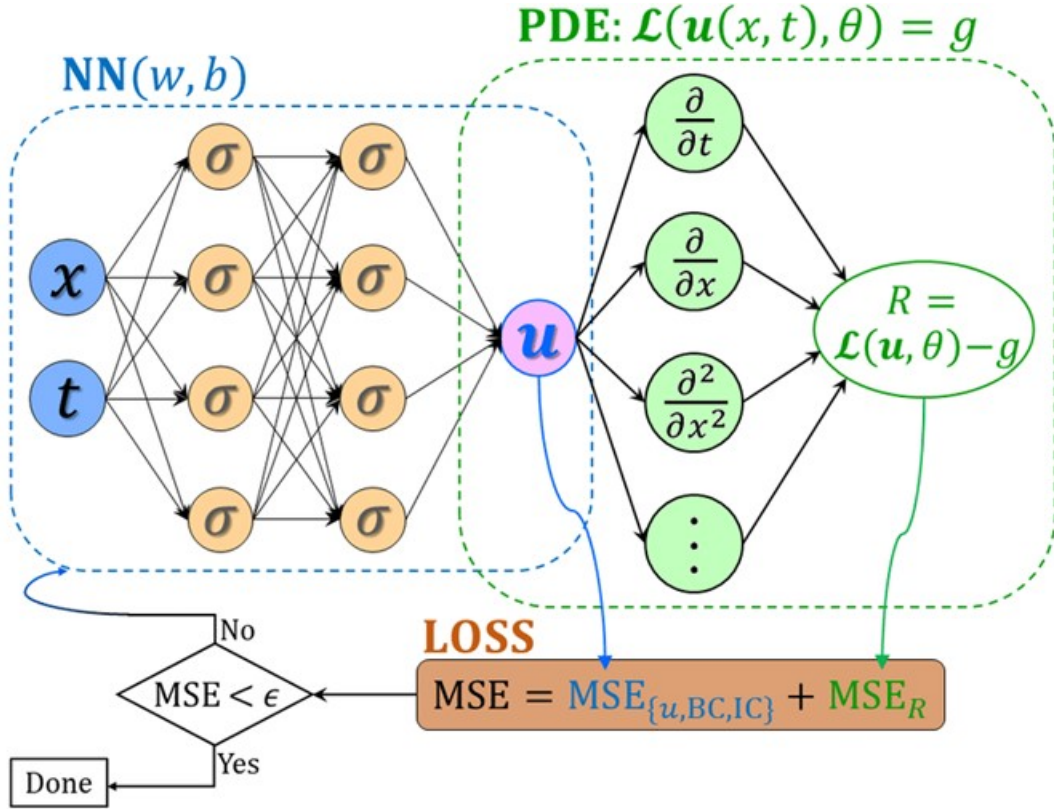


FIGURE 5.5 – La sortie du réseau u est optimisée en minimisant la fonction de perte totale, étant ici une erreur quadratique moyenne (MSE)

On utilise ici la norme L2 relative pour calculer $loss_{pde}$.

En 1D par exemple, on a :

$$\begin{aligned} loss_{pde}(G_\theta(a), a) &= loss_{pde}(\psi_{pred}, a) \\ &= \|ik_0\psi_{pred} + m(x)\psi'_{pred}(x) - g(x)\|_{L^2(\Gamma)} + |\psi_{pred}(0)| \end{aligned}$$

- **Phase d'optimisation (optionnelle)** : Approximation de la solution pour une instance de l'EDP à partir l'opérateur G_θ précédemment appris, en utilisant la fonction de perte $loss_{pde}$.

Dans ce travail, nous nous sommes focalisés sur l'étape d'entraînement, la phase d'optimisation étant surtout nécessaire lorsqu'on s'intéresse aux solutions de l'équation pour des instances particulières. L'algorithme 2 détaille l'étape d'entraînement :

Calcul de la dérivée : Contrairement aux PINNs où il serait possible d'utiliser l'auto-différentiation pour calculer $G'_\theta(a) = \psi'_{pred}$, ce calcul est moins trivial lorsqu'on remplace le réseau de neurones de classique des PINNs par celui utilisé pour le FNO. Ici, nous avons opté pour le calcul de cette dérivée en Fourier.

Algorithm 2 Phase d'entraînement : PINO

Données d'entrée : $\{a_j\}_{j=1}^N = \{x, m_j, Re(g)_j, Im(g)_j\}_{j=1}^N$, $x = (x_k)_{1 \leq k \leq K}$

Données de sortie : $\{Re(\psi_j), Im(\psi_j)\}_{j=1}^N$

Résultat : Opérateur $G_\theta : F \rightarrow U$

- 1: Initialisation de G_θ
- 2: Pour $i = 1, \dots, n_{epochs}$
 - Entrées : $a = \{a_j\}_{j=1}^{batch} = \{x, m_j, Re(g)_j, Im(g)_j\}_{j=1}^{batch}$
 - Calcul de $loss_{total} = \alpha loss_{data}(\psi, G_\theta(a)) + \beta loss_{pde}(G_\theta(a), a)$
 - Optimisation de G_θ par rétropropagation.
 - Génération de nouveaux paramètres(hors base d'apprentissage)
 $a' = \{a'_j\}_{j=1}^{batch}$
 - Calcul de $loss_{pde}(G_\theta(a'), a')$
 - Calcul de $loss_{data}$ en utilisant les instances a
 - $loss_{total} = \alpha loss_{data}(\psi, G_\theta(a)) + \beta loss_{pde}(G_\theta(a'), a')$
 - Optimisation de G_θ par rétropropagation

5.3.3 Résultats**Cas 1D :**

Le tableau 5.6 montre les résultats obtenus. Les paramètres du modèle PINO sont les mêmes que ceux du FNO, à l'exception que pour db1, nous entraînons le modèle sur 3000 observations.

On prendra ici $\alpha = 10$ et $\beta = 1$

TABLE 5.6 – Résultats PINO 1D

Modèle	Données	Instances	loss_train	loss_test
FNO 1D (db1)	$n_{train} = 3000, n_{test} = 2000$	0	0.0117	0.42
PINO 1D (db1)	$n_{train} = 3000, n_{test} = 2000$	100K	0.05	0.076
FNO 1D (db2)	$n_{train} = 10000, n_{test} = 2000$	0	0.02	0.51
PINO 1D (db2)	$n_{train} = 10000, n_{test} = 2000$	200K	0.018	0.38

TABLE 5.7 – Résultats PINO 2D

Modèle	Données	Instances	loss_train	loss_test
FNO 2D (db3)	$n_{train} = 100, n_{test} = 10$	0	0.02	0.65
PINO 2D (db3)	$n_{train} = 100, n_{test} = 10$	1K	0.05	0.6

Note : Le modèle PINO a été entraîné ici sur 100 epochs pour le cas 1D et sur 40 epochs pour le cas 2D. A noter qu'un epoch ici est plus longue qu'un epoch du FNO, dû au fait qu'à chaque itération, nous générons de nouvelles instances de paramètres à chaque epoch afin d'optimiser le modèle. Nous entraînons donc les deux modèles sur un nombre d'épochs différents, mais sur une même durée.

Notons qu'ajouter plus d'instances au modèle a pour but premier d'optimiser l'erreur $loss_{pde}$, mais comme on peut le constater, permet accessoirement d'optimiser notre erreur d'intérêt $loss_{data}$ en 1D, ceci probablement dû à la simplicité de l'équation à résoudre. Pour le cas 2D, cette observation n'est pas forcément vraie et nécessiterait une étude plus poussée.

Chapitre 6

Bilan général

Nous nous sommes intéressés dans le cadre de ce stage aux équations de transport en 1D, et en 2D dans un *shear flow*. L'idée sur le long terme est de pouvoir calculer l'inverse de l'opérateur de transport Goldstein 2.2 et de coupler la méthode utilisée à des *solvers* classiques (pouvant résoudre les autres équations régies par les autres opérateurs présents dans la matrice de 2.1) utilisés en EDPs, afin de résoudre les équations de Goldstein. Comme nous avons pu le constater, l'opérateur neuronal de Fourier (FNO) donne de bons résultats dans les cas étudiés, mais nécessite un besoin élevé de données, ce qui devient d'autant plus critique lorsque les paramètres sont très variables. L'idée de considérer la base de données *db2* en plus de *db1*, vient du fait qu'en pratique, nous ne pouvons pas savoir dans quels intervalles seront ces variables, il est donc utile de regarder différentes situations, et de comprendre ce qui se produit dans chacune d'elles. Hormis le GNO, nous ne nous sommes pas intéressés ici à d'autres architecture pouvant résoudre les dites équations dans un cadre non uniforme, tout en gardant une certaine vitesse d'exécution, bien que certaines méthodes d'interpolation peuvent s'avérer intéressantes. Des travaux très récents [7], s'intéressent d'ailleurs à ce cas de figure, tout en essayant de conserver la rapidité du FNO. De façon globale, l'idée est d'entraîner un réseau de neurones, capable d'apprendre un difféomorphisme entre l'espace physique et un domaine rectangulaire sur lequel pourra être fait une Transformée de Fourier Rapide (FFT). Puisque nous connaissons ici nos équations, il semble plus intéressant de poursuivre les recherches du côté de méthodes de type PINO, où il est possible d'utiliser l'information sur l'EDP dont nous disposons, tout en gardant le bénéfice des opérateurs neuronaux.

Chapitre 7

Conclusion

Le travail que j'ai effectué tout au long de ce stage m'a permis de découvrir et de mieux appréhender divers sujets variés. Tout d'abord, il m'a permis de maîtriser le concept d'apprentissage profond, non seulement d'un point de vue pratique, mais aussi d'un point de vue théorique. Durant mon stage, j'ai pu découvrir le domaine de la physique mathématique en général, et plus particulièrement celui des équations différentielles, qui m'était jusque là quasiment inconnu. J'ai aussi pu accroître de façon considérable mes connaissances en Pytorch et découvrir les multiples fonctionnalités pratiques que propose ce framework.

Ce stage a été pour moi, une expérience très riche sur le plan humain. J'ai eu la chance d'être accueilli dans une *famille*, qui n'a eu de cesse de montrer de la bienveillance envers moi. J'ai pu découvrir le domaine passionnant de la R&D, assister à des réunions d'équipe, quand bien même le sujet étant en dehors de mon champ de compétence. Toutes les difficultés rencontrées au long du stage, que ce soit les *bugs* de code ou les blocages dus à une incompréhension d'un article, ont été une occasion d'apprendre. Depuis le début de mon stage, j'ai ressenti de la fascination et de l'intérêt pour ce sujet sur lequel j'ai travaillé, et ce sentiment n'a pas changé au cours des six derniers mois. Raison pour laquelle je suis fier de pouvoir le continuer en tant que sujet de thèse et de travailler dessus pour les années à venir.

Bibliographie

- [1] Kaushik BHATTACHARYA, Bamdad HOSSEINI, Nikola B. KOVACHKI et Andrew M. STUART. *Model Reduction and Neural Networks for Parametric PDEs*. 2020. DOI : [10.48550/ARXIV.2005.03180](https://doi.org/10.48550/ARXIV.2005.03180). URL : <https://arxiv.org/abs/2005.03180> (cf. p. 12).
- [2] Weinan E et Bing YU. *The Deep Ritz method : A deep learning-based numerical algorithm for solving variational problems*. 2017. DOI : [10.48550/ARXIV.1710.00211](https://doi.org/10.48550/ARXIV.1710.00211). URL : <https://arxiv.org/abs/1710.00211> (cf. p. 11).
- [3] Justin GILMER, Samuel S. SCHOENHOLZ, Patrick F. RILEY, Oriol VINYALS et George E. DAHL. **Neural Message Passing for Quantum Chemistry**. *CoRR* abs/1704.01212 (2017). arXiv : [1704.01212](https://arxiv.org/abs/1704.01212). URL : <http://arxiv.org/abs/1704.01212> (cf. p. 27, 28).
- [4] William H. GUSS. *Deep Function Machines : Generalized Neural Networks for Topological Layer Expression*. 2016. DOI : [10.48550/ARXIV.1612.04799](https://doi.org/10.48550/ARXIV.1612.04799). URL : <https://arxiv.org/abs/1612.04799> (cf. p. 12).
- [5] Dan HENDRYCKS et Kevin GIMPEL. **Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units**. *CoRR* abs/1606.08415 (2016). arXiv : [1606.08415](https://arxiv.org/abs/1606.08415). URL : <http://arxiv.org/abs/1606.08415> (cf. p. 40).
- [6] Nicolas LE ROUX et Yoshua BENGIO. **Continuous Neural Networks**. In : *AISTAT2007*. Jan. 2007. URL : <https://www.microsoft.com/en-us/research/publication/continuous-neural-networks/> (cf. p. 12).
- [7] Zongyi LI, Daniel Zhengyu HUANG, Burigede LIU et Anima ANANDKUMAR. *Fourier Neural Operator with Learned Deformations for PDEs on General Geometries*. 2022. DOI : [10.48550/ARXIV.2207.05209](https://doi.org/10.48550/ARXIV.2207.05209). URL : <https://arxiv.org/abs/2207.05209> (cf. p. 34).

- [8] Zongyi LI, Nikola B. KOVACHKI, Kamyar AZIZZADENESHELI, Burigede LIU, Kaushik BHATTACHARYA, Andrew M. STUART et Anima ANANDKUMAR. **Fourier Neural Operator for Parametric Partial Differential Equations.** *CoRR* abs/2010.08895 (2020). arXiv : [2010.08895](https://arxiv.org/abs/2010.08895). URL : <https://arxiv.org/abs/2010.08895> (cf. p. 12, 21, 29).
- [9] Zongyi LI, Nikola B. KOVACHKI, Kamyar AZIZZADENESHELI, Burigede LIU, Kaushik BHATTACHARYA, Andrew M. STUART et Anima ANANDKUMAR. **Multipole Graph Neural Operator for Parametric Partial Differential Equations.** *CoRR* abs/2006.09535 (2020). arXiv : [2006.09535](https://arxiv.org/abs/2006.09535). URL : <https://arxiv.org/abs/2006.09535> (cf. p. 12, 29).
- [10] Zongyi LI, Nikola B. KOVACHKI, Kamyar AZIZZADENESHELI, Burigede LIU, Kaushik BHATTACHARYA, Andrew M. STUART et Anima ANANDKUMAR. **Neural Operator : Graph Kernel Network for Partial Differential Equations.** *CoRR* abs/2003.03485 (2020). arXiv : [2003.03485](https://arxiv.org/abs/2003.03485). URL : <https://arxiv.org/abs/2003.03485> (cf. p. 12, 20, 27, 29).
- [11] Zongyi LI, Hongkai ZHENG, Nikola B. KOVACHKI, David JIN, Haoxuan CHEN, Burigede LIU, Kamyar AZIZZADENESHELI et Anima ANANDKUMAR. **Physics-Informed Neural Operator for Learning Partial Differential Equations.** *CoRR* abs/2111.03794 (2021). arXiv : [2111.03794](https://arxiv.org/abs/2111.03794). URL : <https://arxiv.org/abs/2111.03794> (cf. p. 12, 30).
- [12] Lu LU, Pengzhan JIN et George Em KARNIADAKIS. **DeepONet : Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators.** *CoRR* abs/1910.03193 (2019). arXiv : [1910.03193](https://arxiv.org/abs/1910.03193). URL : <http://arxiv.org/abs/1910.03193> (cf. p. 12).
- [13] Nicholas H. NELSEN et Andrew M. STUART. **The Random Feature Model for Input-Output Maps between Banach Spaces.** *SIAM Journal on Scientific Computing* 43 :5 (jan. 2021), A3212-A3243. DOI : [10.1137/20m133957x](https://doi.org/10.1137/20m133957x). URL : <https://doi.org/10.1137/20m133957x> (cf. p. 12).
- [14] Ravi G. PATEL, Nathaniel A. TRASK, Mitchell A. WOOD et Eric C. CYR. **A physics-informed operator regression framework for extracting data-driven continuum models.** *Computer Methods in Applied Mechanics and Engineering* 373 (jan. 2021), 113500. DOI : [10.1016/j.cma.2020.113500](https://doi.org/10.1016/j.cma.2020.113500). URL : <https://doi.org/10.1016/j.cma.2020.113500> (cf. p. 12).
- [15] M. RAISSI, P. PERDIKARIS et G.E. KARNIADAKIS. **Physics-informed neural networks : A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations.** *Journal of Computational Physics* 378 (2019), 686-707. ISSN : 0021-9991. DOI : <https://doi.org/10.1016/j.jcp.2018.10.045>. URL :

<https://www.sciencedirect.com/science/article/pii/S0021999118307125>

(cf. p. 11, 29).

- [16] Christopher K. I. WILLIAMS. **Computing with Infinite Networks**. In : *NIPS*. 1996, 295-301. URL : <http://papers.nips.cc/paper/1197-computing-with-infinite-networks> (cf. p. 12).

Chapitre 8

Annexe

Table des notations

TABLE 8.1 – Table des notations

Notations	Significations
$D \subset \mathbb{R}^d$	Domaine de l'EDP
Γ	Ensemble de points de la discrétisation de D
σ	Fonction d'activation
ν	Mesure de Borel
W, P, Q, p, q	Paramètres appris par le modèle
K	Nombre de points de la discrétisation
lr	Learning rate
d_v	Dimension dans laquelle P projète les données d'entrée
$t = 0, \dots, T$	Couches du modèle
k_{max}	Nombre de modes de Fourier conservés
n_{train}	Nombre de données d'entraînement
n_{test}	Nombre de données de test

Discrétisation spatiale

La discrétisation spatiale permet de transformer une équation aux dérivées partielles en une équation matricielle. Discrétiser un espace permet de faire des simulations de calcul et des représentations graphiques.

Ecoulement

On entend par écoulement le déplacement de l'air par rapport à un objet. On dénombre trois types d'écoulement, l'**écoulement laminaire** (aucun

échange entre les particules d'air qui suivent un mouvement rectiligne, l'écoulement turbulent (aucun échange entre les particules d'air, mais leur mouvement n'est plus rectiligne) et l'**écoulement tourbillonnaire** (les particules d'air se mélangent et ne suivent aucun mouvement particulier).

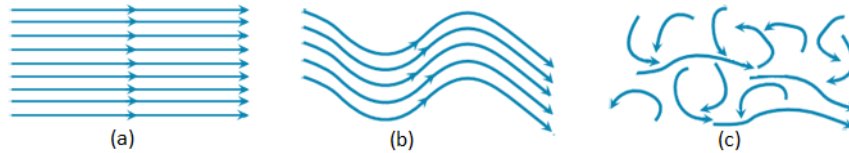


FIGURE 8.1 – (a) : Ecoulement laminaire, (b) : Ecoulement turbulent, (c) : Ecoulement tourbillonnaire

La fonction ReLU

La ReLU (Rectified Linear Unit) est une fonction non linéaire très utilisée en apprentissage profond, permettant de remplacer les résultats négatifs par 0. Elles permettent un entraînement plus rapide comparé aux fonctions sigmoïd et tanh de par sa sparsité représentationnelle. L'écriture mathématique de la ReLU est la suivante :

Soit $x \in \mathbb{R}$,

$$\text{ReLU}(x) = \max(0, x)$$

La fonction GELU

La GELU (Gaussian Error Linear Unit) [5], comme la ReLU, fait partie des fonctions d'activation utilisées en apprentissage profond.

Pour x donnée,

$$\text{GELU}(x) = xP(X) \text{ où } X \sim \mathcal{N}(0, 1)$$

Contrairement aux autres fonctions d'activation (ReLU, ELU, PReLU), la GELU en plus de la non linéarité, apporte aussi un effet de régularisation.

La transformée de Fourier

La transformée de Fourier est une opération qui vise à transformer une fonction intégrable en une autre fonction. Elle est généralement notée par $\mathcal{F}$ et s'écrit :

Soit f une fonction intégrable,

$$\mathcal{F}(f) : \xi \mapsto \int_{-\infty}^{+\infty} f(x) e^{-i\xi x} dx$$

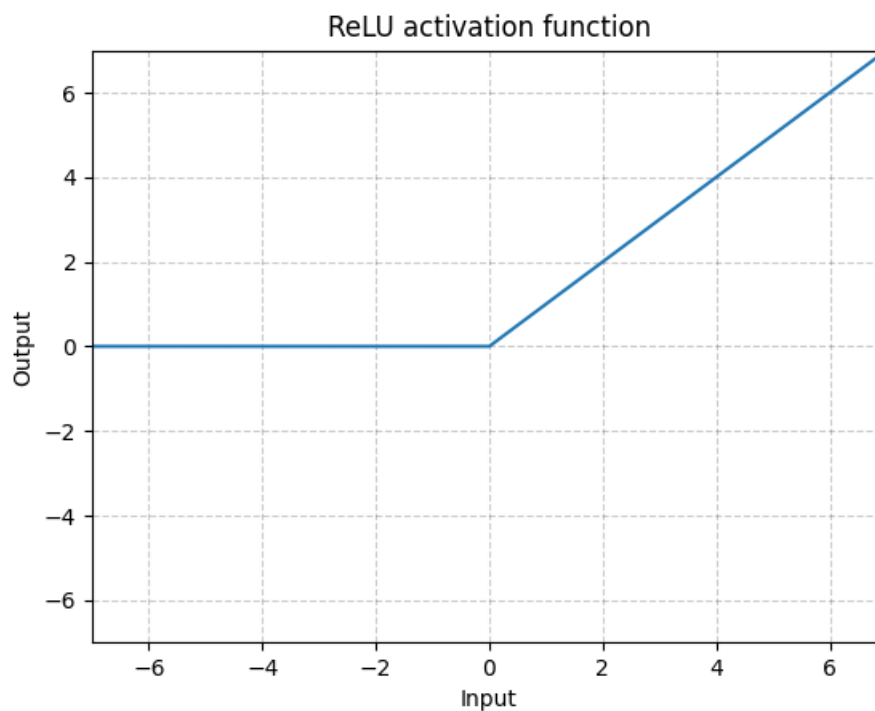


FIGURE 8.2 – Rectified Linear Unit (ReLU)

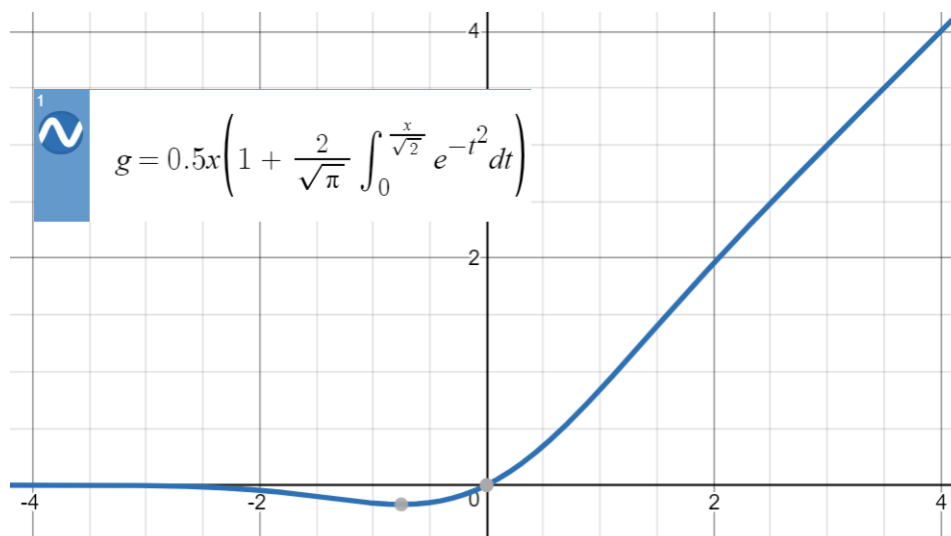


FIGURE 8.3 – Gaussian Error Linear Unit (GELU)

Elle est souvent aussi écrite de la manière suivante :

$$\mathcal{F}(f) : \xi \mapsto \int_{-\infty}^{+\infty} f(x) e^{-i2\pi\xi x} dx$$

La transformée de Fourier inverse s'écrit :

$$\mathcal{F}(\hat{f}) : \xi \mapsto \int_{-\infty}^{+\infty} \hat{f}(x) e^{+i\xi x} dx$$

Pour un signal f de N échantillons, la Transformée de Fourier Discrète (TFD) s'écrit :

$$f(k) = \sum_{n=0}^{N-1} f(n) e^{-2i\pi k \frac{n}{N}}$$

La classe Parameter

La classe Parameter de Pytorch, très utilisée en apprentissage profond, notamment pour les réseaux de neurones récurrents (RNNs) permet d'initialiser un tenseur en tant que variable, et qui sera considéré comme paramètre à entraîner par le modèle.